



UNIVERSITÉ PARIS 1 — PANTHÉON-SORBONNE
MASTER MIMO

Projet Data en Python

Analyse du fichier BAAC — accidents de la route 2005–2015

Présenté par :
Rayan Leveque
Tom Séguier

Sous la direction de :
M. Grosz
M. Giron

En-tête

- **Membres du groupe** : Rayan Leveque, Tom Séguier
- **Période étudiée** : 2005 – 2015 inclus (Groupe 1, 11 années)
- **Outil d'IA générative utilisé** : **Claude Code** (Anthropic), modèle Claude Opus 4.7 (1M context)

Note sur l'usage de l'outil

Claude Code est un **agent IA conversationnel** doté d'outils (lecture/écriture de fichiers, exécution de commandes shell, etc.), et non un LLM one-shot type ChatGPT. Le workflow n'a donc pas reposé sur des prompts uniques copiables, mais sur une **conversation itérative** avec l'agent, intégrée à l'environnement de développement (terminal).

En conséquence, la rubrique "prompt utilisé" du barème de référence (§1.4.1 du sujet) est remplacée dans ce dossier par une rubrique "**Travail avec l'agent**", qui décrit pour chaque question :

- Le **contexte donné à l'agent** : contraintes, données d'entrée, format de sortie attendu.
- Les **itérations et décisions** : ce qui a été ajusté, ce qui a posé problème, ce qui a été validé.

Cette approche est honnête sur le workflow réel, et l'analyse de la stratégie d'usage d'un agent IA (vs prompting one-shot) est développée dans l'exposé.

Table des matières

En-tête	1
Note sur l'usage de l'outil	1
1. Récupération et formatage des données	3
Question 0.0 — Téléchargement des données BAAC	4
Question 1.1 — Conversion UTF-8, séparateur ,	5
Question 1.2 — Vérification de structure et tableaux récapitulatifs	7
Question 1.3 — Enrichissement avec libellés des domaines énumérés	9
2. Étude de la qualité des données	10
2.1. Intégrité référentielle	10
Question 2.1 — Usager → Véhicule	11
Question 2.2 — Véhicule → ≥ 1 Usager	13
Question 2.3 (<i>facultative</i>) — Lieu → Accident	15
Question 2.4 (<i>facultative</i>) — Accident → un seul Lieu	17
2.2. Domaines énumérés	18
Question 3.1 — Répartition % des attributs énumérés (usagers)	19
2.3. Valeurs absentes	20
Question 4.1 — Taux de valeurs absentes (véhicules)	21
2.4. Cohérence inter-attributs	22
Question 5.1 — Incohérence $lum \times hrnm$	23
Question 5.2 — Autre incohérence inter-attributs ($obsn \times choc$)	25
Question 5.3 (<i>facultative</i>) — Troisième incohérence ($an_nais \times période$)	27
3. Requêtage et visualisation	28
Question 6.1 — Accidents par tranche horaire d'1h	29
Question 6.2 — Tués cumulés par tranche d'âge et sexe	31

Question 6.3 — Top 10 départements (carte folium)	33
Question 6.4 — Question libre	36
Question 6.5 (<i>facultative</i>) — Carte animée par année	39
Conclusion / synthèse	41

1. Récupération et formatage des données

Question 0.0 — Téléchargement des données BAAC

Rappel de la question : récupérer les fichiers caractéristiques, lieux, véhicules, usagers pour les années 2005–2015, depuis data.gouv.fr, ainsi que le PDF de description des bases.

Travail avec l'agent — Claude Code

- **Contexte** : automatiser le téléchargement plutôt que de cliquer 45 fois sur des liens, en s'assurant d'exclure les `vehicules-immatricules-baac...` (piège du sujet).
- **Itérations** : première version basée sur les titres des ressources de l'API data.gouv.fr; le filtre ratait `caracteristiques` à cause des accents (titres en `caractéristiques`), corrigé en normalisant via `unicodedata` et en matchant aussi le nom de fichier de l'URL.

Fichier Python : `python/download_data.py` (*hors barème — script utilitaire de setup, pas une réponse à une question notée*).

Résultat : ~44 fichiers CSV + 1 PDF déposés dans `data/raw/`.

Vérifications. `ls data/raw/ | wc -l` confirme le compte attendu ($4 \text{ types} \times 11 \text{ ans} + 1 \text{ PDF}$). Échantillonnage manuel : ouverture de 2-3 CSV pour confirmer qu'ils correspondent bien au type annoncé.

Question 1.1 — Conversion UTF-8, séparateur ,

Rappel de la question. Par précaution, transformer le codage de tous les fichiers de données au format UTF-8 et s'assurer que le séparateur est bien ,.

Travail avec l'agent — Claude Code

- **Contexte donné à l'agent** : 4 types $\times$ 11 ans = 44 fichiers dans `data/raw/`, sortie attendue dans `data/utf8/` avec nommage `{type}_{année}.csv`, code en français, boucles explicites, prints de progression.
- **Stratégie validée** : (1) détection d'encodage par `chardet` sur un échantillon de 200 ko, (2) détection de séparateur en comptant , , ; et `\t` dans la ligne d'en-tête, (3) relecture via `csv.reader` et réécriture via `csv.writer` (qui gère proprement le quoting si une virgule apparaît dans un champ texte).
- **Itération principale** : avant exécution, j'ai inspecté manuellement quelques fichiers (`file -b`, `od -c`) pour confirmer ce que la détection allait trouver — c'est ce qui m'a fait découvrir le cas particulier de `caracteristiques_2009.csv` (séparateur tabulation au lieu de virgule), qui justifie la détection automatique du séparateur plutôt qu'un simple `read/write UTF-8`.

Fichier Python : `python/question11.py`.

Vérification (pandas) — `python/verification11.py` → `results/verification11.txt` :

```
>>> for f in utf8.glob("*.csv"): # 44 fichiers
... f.read_bytes().decode("utf-8") # decode UTF-8 OK
... pd.read_csv(f, sep=",").shape[1] # == nb colonnes attendu
>>> recap

      type fichiers utf8+sep_ok
caracteristiques 11 11
      lieux      11 11
    vehicules    11 11
      usagers    11 11

# conforme : 44/44 fichiers UTF-8, sep ',', bon nb de colonnes
```

Résultat : `results/reponse11.txt` (44 fichiers convertis, 0 manquant). Synthèse des encodages détectés : 34 fichiers déjà en UTF-8 (lieux, vehicules, usagers de toutes années), 9 fichiers en `iso-8859-1` (`caracteristiques` 2005, 2007–2015), 1 fichier en `windows-1252` (`caracteristiques` 2006). Séparateurs : 43 fichiers en , , 1 fichier en tabulation (`caracteristiques` 2009).

Vérifications.

- *Comptes de lignes* : `wc -l` sur les fichiers source et cible donne le même résultat pour chaque fichier (ex. 87 027 lignes pour `caracteristiques_2005.csv` source et cible) — preuve qu'aucune ligne n'a été perdue.
- *Encodage* : `file data/utf8/caracteristiques_2009.csv` renvoie `CSV Unicode text, UTF-8 text` (idem pour les fichiers initialement en latin-1).
- *Caractères accentués* : `grep -E "É|é|è|ç"` sur `caracteristiques_2005.csv` (ex-`iso-8859-1`) renvoie des libellés correctement décodés (Théodore Blot, libération, intermarché).
- *Cas 2009* : le fichier source contenait déjà des caractères de remplacement Unicode (`U+FFFD`, vus comme ?) provenant d'une corruption en amont chez le producteur des données — la conversion les conserve à l'identique, on ne peut pas les reconstruire ; cela sera signalé comme une limite des données dans la conclusion.
- *Quoting CSV* : le passage par `csv.writer` garantit que si un champ contient , , il sera entouré de guillemets — c'est plus robuste qu'un simple `replace('\t', ',')`.

Commentaires. La diversité d'encodages (3 encodages distincts) et le séparateur tabulation isolé en 2009 confirment que la précaution demandée par l'énoncé est nécessaire : sans cette étape, un script Q2+ qui lirait `caracteristiques_2009.csv` en supposant `utf-8 +`, casserait silencieusement (lignes considérées comme une seule colonne).

Question 1.2 — Vérification de structure et tableaux récapitulatifs

Rappel de la question. Vérifier par programme que pour chaque type de fichier (caractéristiques, lieux, véhicules, usagers), les fichiers des différentes années ont une structure conforme à la documentation. Produire 4 tableaux récapitulatifs avec le nombre de lignes par année et le cumul pour chaque type.

Travail avec l'agent — Claude Code

- **Contexte donné à l'agent** : entrée dans `data/utf8/` (sortie de Q1.1), schéma de référence pour les 4 types de fichiers tel qu'observé dans la doc BAAC + les en-têtes 2005-2015 (caracteristiques 16 colonnes, lieux 18, vehicules 9, usagers 12).
- **Sortie attendue** : `results/reponse12.txt` avec 4 tableaux formatés (année, nb_lignes, nb_colonnes, cumul) plus une synthèse des écarts.
- **Choix d'implémentation** : `csv.reader` ligne à ligne pour le comptage (au lieu de `pandas` — un script qui lit du CSV proprement avec la lib standard est plus pédagogique pour la soutenance), comparaison des en-têtes par ensemble *et* par ordre pour distinguer trois types d'anomalies (colonne manquante, colonne inattendue, ordre incorrect).
- **Itération mineure** : avant lancement, j'ai inspecté manuellement les en-têtes via `head -1` sur les 44 fichiers pour confirmer que le schéma resterait stable sur toute la période — c'est ce qui m'a permis d'écrire un schéma unique par type plutôt qu'un schéma par (type, année).

Fichier Python : `python/question12.py`.

Vérification (`pandas`) — `python/verification12.py` → `results/verification12.txt` :

```
>>> df = pd.read_csv(f, dtype=str)
>>> list(df.columns) == schema_attendu # en-tete exact ?
>>> recap # nb de lignes par type

      type colonnes schema_ok total_lignes
caracteristiques 16 11/11 780553
      lieux 18 11/11 780553
vehicules 9 11/11 1331465
usagers 12 11/11 1742583

# conforme : en-tete exact (colonnes+ordre) pour les 44 fichiers
```

Résultat : `results/reponse12.txt`. Cumul par type pour 2005-2015 : caracteristiques **780 553**, lieux **780 553** (cohérent avec caracteristiques — 1 ligne lieu par accident), vehicules **1 331 465**, usagers **1 742 583**. Cumul global : **4 635 154 lignes** de données. Aucun écart de schéma : les 44 fichiers ont exactement les colonnes attendues, dans le bon ordre.

Vérifications.

- *Cohérence avec Q1.1* : Q1.1 comptait `nb_lignes` en incluant l'en-tête (87 027 pour `caracteristiques_2005.csv`), Q1.2 compte les lignes de données seules (87 026). L'écart constant de 1 ligne par fichier confirme qu'aucune ligne de données n'a été perdue lors de la conversion UTF-8.
- *Égalité caracteristiques ↔ lieux par année* : pour chaque année, le nombre de lignes de `caracteristiques` est strictement égal à celui de `lieux` (87 026 = 87 026 en 2005, etc.)
 - première vérification de la relation 1-pour-1 entre un accident et son lieu, qui sera approfondie en Q2.4.
- *Ratios* : le ratio `usagers / caracteristiques` est de 2,23 sur la période (4 635 154 / 780 553).

553 $\rightarrow$ en moyenne $\approx 2,23$ usagers par accident), et `vehicules / caracteristiques` $\approx 1,71$ — ordres de grandeur plausibles pour des accidents de la circulation, où la majorité implique 2 véhicules.

- *Schéma stable sur 11 ans* : aucune colonne ajoutée ou retirée entre 2005 et 2015 pour les 4 types — la doc BAAC dont on s’est servi décrit donc bien la totalité du périmètre du Groupe 1.

Commentaires. L’absence d’écart de schéma est une bonne nouvelle pour la suite : les scripts Q2+ pourront supposer un en-tête fixe par type. À noter que la baisse du nombre d’accidents sur la période (87 k en 2005 $\rightarrow$ 58 k en 2015, soit -33%) est visible directement dans le cumul des `caracteristiques` ; c’est cohérent avec la tendance nationale connue de baisse de l’accidentologie routière, et cela sera repris en Q6.1 (analyse temporelle).

Question 1.3 — Enrichissement avec libellés des domaines énumérés

Rappel de la question. Pour tous les fichiers obtenus en 1.1, générer une nouvelle version où toutes les valeurs des domaines énumérés sont complétées avec le libellé correspondant (ex. 1 pour lum devient 1 - Plein jour). Vérifier la transformation, en portant une attention particulière à catv (catégorie de véhicule).

Travail avec l'agent — Claude Code

- **Contexte donné à l'agent** : 26 colonnes énumérées réparties sur les 4 types de fichiers (lum, agg, int, atm, col pour caractéristiques; catr, circ, vosp, prof, plan, surf, infra, situ pour lieux; senc, catv, obs, obsm, choc, manv pour véhicules; catu, grav, sexe, trajet, locp, actp, etatp pour usagers). Sortie attendue dans data/enriched/ (même nommage), au format code - libellé.
- **Source des libellés** : la doc PDF ONISR 2005-2018 et docs/schema_baac.md.
- **Choix d'implémentation** : un dictionnaire Python par colonne énumérée (clé = code en chaîne, valeur = libellé français), un dispatch MAPPINGS[type][colonne], et un parcours csv.reader / csv.writer ligne à ligne.
- **Itération principale** : la première version traitait toutes les colonnes comme du texte trim simple, mais les codes catv, obs, manv sont stockés zero-paddés sur 2 chiffres dans les fichiers (07, 01, 30...) — j'ai donc ajouté un normaliser_code() qui applique zfill(2) uniquement à ces 3 colonnes avant lookup.
- **Décision validée** : ne pas enrichir les valeurs absentes ("", '.', -1) ni les codes hors mapping — on les laisse à l'identique pour les analyser proprement en Q4 (taux d'absents) plutôt que de fabriquer un faux libellé.

Fichier Python : python/question13.py.

Vérification (pandas) — python/verification13.py → results/verification13.txt :

```
>>> ok = col.str.contains("- ") | col.isin(MARQUEURS) # enrichi/absent
>>> recap # lignes conservees + hors mapping

      type lignes_utf8 lignes_enriched hors_mapping
caracteristiques 780553 780553 0
      lieux 780553 780553 0
      vehicules 1331465 1331465 0
      usagers 1742583 1742583 0

# conforme : aucune ligne perdue, 0 valeur hors mapping, catv 33 libelles
```

Résultat : 44 fichiers enrichis dans data/enriched/ (rapport détaillé dans results/reponse13.txt). Cumul sur 2005-2015 : ~21,4 millions de cellules enrichies. Sur la colonne **catv** (1 331 465 lignes véhicules), **100 % des codes rencontrés sont répertoriés** (33 codes distincts, du plus fréquent 07 - VL seul à 825 576 occurrences au plus rare 11 - VU + caravane à 17 occurrences). Aucun code catv n'est donc orphelin du mapping — le piège mentionné dans le sujet est neutralisé.

Vérifications.

- *Comptage de lignes* : wc -l sur source vs cible donne le même résultat (ex. 69 380 lignes pour caracteristiques_2010.csv source et cible) — aucune ligne perdue.
- *Couverture catv* : la rubrique "4. Vérification ciblée" du rapport balaye les 11 années de vehicules_*.csv enrichis et confirme que les 33 codes rencontrés sont tous étiquetés enrichi (aucun non répertorié).

- *Inspection visuelle* : `head -3 data/enriched/{caracteristiques,lieux,vehicules,usagers}_2010` montre des libellés propres (ex. 1 - Plein jour, 30 - Scooter ≤ 50 cm³, 0 - Sans objet).
- *Colonnes non-énumérées préservées* : Num_Acc, dep, voie, secu, an_nais, etc. apparaissent inchangées dans les fichiers enrichis — la transformation est strictement ciblée.
- *Cellules "hors mapping"* : 1,67 M cellules au total (essentiellement `senc=0` pour les 2 années où le sens de circulation est non renseigné, `manv=00`, et `circ/prof/plan/surf/situ=0` dans `lieux`). Toutes correspondent au code 0 non documenté dans la doc PDF mais utilisé en pratique comme marqueur "non renseigné". Décision : on les laisse à 0 brut. Cela sera repris en Q4 (où on les compte comme absentes) et en Q5 (où elles n'interfèrent pas avec les contrôles d'incohérence).

Commentaires. L'enrichissement remplit deux rôles : (1) rendre les fichiers `data/enriched/` directement lisibles par un humain (utile en debug et pour générer les tableaux narratifs des questions ultérieures), (2) servir de base unique aux Q2-Q6 — toutes ces questions liront depuis `data/enriched/` plutôt que `data/utf8/`, ce qui garantit que les répartitions calculées en Q3.1 et les vérifications de cohérence en Q5 utiliseront exactement les mêmes libellés que ceux qui apparaîtront dans le rapport. Le format `code - libellé` (plutôt que le libellé seul) préserve la possibilité de retrier ou regrouper sur le code numérique d'origine si nécessaire — c'est moins coûteux que de re-décoder en sens inverse.

2. Étude de la qualité des données

2.1. Intégrité référentielle

Question 2.1 — Usager → Véhicule

Rappel de la question. Vérifier par programme que chaque usager fait référence à un véhicule présent dans les véhicules. Les données incohérentes doivent être listées dans un fichier dédié, analysées et commentées.

Travail avec l'agent — Claude Code

- **Contexte donné à l'agent** : entrée dans `data/enriched/` (sortie de Q1.3), clé de jointure (`Num_Acc`, `num_veh`) — un usager pointe vers le véhicule auquel il est rattaché (conducteur/passager) ou qui l'a heurté (piéton).
- **Sortie attendue** : `results/reponse21.txt` (résumé annuel + analyse) et `results/reponse21_violations.csv` (liste exhaustive des violations avec colonnes `annee`, `Num_Acc`, `num_veh`).
- **Choix d'implémentation** : traitement année par année — pour chaque année on charge l'ensemble des couples (`Num_Acc`, `num_veh`) du fichier `vehicules_{annee}.csv` dans un `set` Python, puis on parcourt `usagers_{annee}.csv` ligne à ligne en testant l'appartenance. Le `set` est en $O(1)$ à la lecture, le tout coûte $O(n)$ sur les ~1,7 M usagers et tient largement en mémoire (~150 k clés par année).
- **Itération mineure** : avant d'écrire le code, j'ai inspecté les en-têtes des deux fichiers (`head -1`) pour confirmer que `num_veh` est bien la colonne de jointure et qu'elle existe des deux côtés ; les en-têtes confirment `Num_Acc`, ..., `num_veh` côté `vehicules` et `Num_Acc`, ..., `num_veh` côté `usagers` — la jointure est directe sans renommage.

Fichier Python : `python/question21.py`.

Vérification (pandas) — `python/verification21.py` → `results/verification21.txt` :

```
>>> f = usagers.merge(vehicules, on=["Num_Acc", "num_veh"],
... how="left", indicator=True)
>>> recap # violations = lignes "left_only" par annee

annee usagers couples_veh violations
2005 197498 149764 2
2006 187085 142300 5
2007 188457 144130 11
2008 170960 131048 4
2009 165962 126347 0
2010 154192 117743 0
2011 148543 113854 0
2012 138628 105814 0
2013 128694 98925 0
2014 132186 101762 0
2015 130378 99778 0

# conforme : 22 violations / 13 couples distincts (identique a reponse21_violations.csv)
```

Résultat : `results/reponse21.txt` + `results/reponse21_violations.csv`. Sur **1 742 583 usagers** cumulés sur 2005-2015, **22 violations** détectées, soit **0,0013 %** — toutes concentrées sur 2005-2008 (2, 5, 11, 4 violations respectivement) ; **à partir de 2009 l'intégrité est parfaite** (0 violation chaque année). Aucune violation n'est due à un `num_veh` vide : il s'agit à chaque fois d'un identifiant non vide pointant vers un véhicule inexistant pour le `Num_Acc` concerné.

Vérifications.

- *Cohérence des totaux* : la somme des `nb_usagers` par année (1 742 583) coïncide exacte-

ment avec le cumul calculé en Q1.2 — preuve que toutes les lignes sont bien balayées.

- *Inspection manuelle de violations* : sur l'accident 200500086196 (2005), le fichier `vehicules` n'enregistre que A01 et B02, mais un piéton est saisi avec `num_veh=A03` — le saisisseur a probablement noté la lettre du véhicule heurtant sans vérifier l'enregistrement côté véhicules. Sur 200700083469 (2007), le fichier `vehicules` contient A01 (VL) et B01 (autobus) ; les 9 passagers du bus sont ventilés entre B01 et B02, alors qu'aucun véhicule B02 n'a été déclaré — visiblement, le saisisseur a inventé un second véhicule pour répartir les passagers. Ces deux exemples confirment l'origine *humaine* (saisie BAAC sur procès-verbal) des erreurs.
- *Réciprocité non-implicite* : ce contrôle ne dit rien des véhicules sans usager (cas inverse) — c'est l'objet de Q2.2.
- *Échantillon vs exhaustif* : le fichier CSV contient bien les 22 violations (`wc -l reponse21_violations.csv` = 23 = 22 + en-tête).

Commentaires. Le taux de violations (0,0013 %) est négligeable et ne biaise aucune analyse statistique ultérieure. La concentration sur 2005-2008 puis disparition totale à partir de 2009 suggère soit un durcissement du contrôle d'intégrité dans la chaîne de saisie BAAC autour de 2008-2009, soit un nettoyage rétroactif partiel des fichiers récents. Les violations restantes seront simplement ignorées dans les questions suivantes ; la liste exhaustive reste disponible dans `reponse21_violations.csv` pour qui voudrait les exclure explicitement par jointure.

Question 2.2 — Véhicule $\rightarrow \geq 1$ Usager

Rappel de la question. Vérifier par programme que chaque véhicule décrit dans les véhicules est associé à au moins un usager. Si des données incohérentes existent, elles doivent être toutes listées dans un fichier dédié, analysées et commentées.

Travail avec l'agent — Claude Code

- **Contexte donné à l'agent** : c'est le contrôle réciproque de Q2.1 — même clé de jointure (`Num_Acc`, `num_veh`), mais on inverse le sens (on parcourt `vehicules_*.csv` et on vérifie l'existence côté `usagers_*.csv`).
- **Sortie attendue** : `results/reponse22.txt` (résumé annuel + analyse) et `results/reponse22_violations.csv` (`annee`, `Num_Acc`, `num_veh`).
- **Choix d'implémentation** : structure miroir de `question21.py` — pour chaque année on charge l'ensemble des couples (`Num_Acc`, `num_veh`) du fichier `usagers_{annee}.csv` dans un `set`, puis on parcourt `vehicules_{annee}.csv` ligne à ligne en testant l'appartenance. Le `set` côté `usagers` est un peu plus lourd (~140 k clés/année vs ~120 k pour `vehicules`) mais cela tient largement en mémoire.
- **Itération principale** : avant de rédiger l'analyse, j'ai voulu *valider la cause supposée* — j'ai regardé manuellement le premier cas du CSV de violations (accident 200500000170, 2005) : côté `vehicules`, on a un cycliste A01 et un VU B02 ; côté `usagers`, seul le cycliste apparaît comme conducteur. Le VU n'a aucun usager rattaché — c'est exactement la signature d'une fuite. L'analyse écrite en partie 3 du rapport mentionne donc explicitement ce scénario plutôt que de parler vaguement d'"erreurs de saisie".

Fichier Python : `python/question22.py`.

Vérification (pandas) — `python/verification22.py` $\rightarrow$ `results/verification22.txt` :

```
>>> f = vehicules.merge(usagers, on=["Num_Acc", "num_veh"],
... how="left", indicator=True)
>>> recap # vehicules sans usager par annee

annee vehicules couples_usa violations
2005 149764 147779 1987
2006 142300 140264 2040
2007 144130 141975 2158
2008 131048 129052 2000
2009 126347 124708 1639
2010 117743 116141 1602
2011 113854 112211 1643
2012 105814 104217 1597
2013 98925 97348 1577
2014 101762 100063 1699
2015 99778 98140 1638

# conforme : 19 580 vehicules sans usager (identique)
```

Résultat : `results/reponse22.txt` + `results/reponse22_violations.csv`. Sur **1 331 465 véhicules** cumulés sur 2005-2015, **19 580 véhicules sans usager** (1,4706 %), répartis de manière régulière sur toute la période (1577 à 2158 par année, sans rupture franche entre les années anciennes et récentes — contrairement à Q2.1). Aucun `num_veh` vide côté véhicules : la clé est toujours bien renseignée, le problème ne vient pas de la saisie de l'identifiant mais bien de l'absence d'un usager rattaché.

Vérifications.

- *Cohérence des totaux* : la somme des **nb_vehicules** (1 331 465) coïncide exactement avec le cumul **vehicules** calculé en Q1.2 — preuve que toutes les lignes sont bien balayées.
- *Inspection manuelle d'un cas* : sur l'accident 200500000170 (2005), **vehicules** enregistre A01 (bicyclette) et B02 (VU 1,5T-3,5T) ; côté **usagers**, seul le cycliste (A01, blessé léger, conducteur) est saisi — aucun usager pour le VU B02. Sans information sur le conducteur du VU, le scénario le plus crédible est une fuite après collision avec le cycliste, où le véhicule a été identifié (par le cycliste, des témoins, ou la plaque) sans que son conducteur ne soit retrouvé au moment du PV. Cela colle au cas connu signalé dans l'énoncé du sujet.
- *Stationnarité dans le temps* : contrairement à Q2.1 (concentration sur 2005-2008), le taux ici est stable sur 11 ans (~1,5 % chaque année, taux le plus bas en 2013 = 1,59 %, le plus haut en 2007 = 1,50 %). Ce profil "bruit de fond constant" est cohérent avec un phénomène structurel (fuites de conducteurs) et non avec une erreur de saisie ponctuelle qui aurait été corrigée — preuve indirecte supplémentaire de l'interprétation "fuite".
- *Réciprocité* : le contrôle Q2.1 portait sur **usagers** → **vehicules** (22 violations) ; ici c'est l'inverse **vehicules** → **usagers** (19 580 violations). Les deux ne couvrent pas le même problème — Q2.1 attrape les usagers fantômes (saisie incorrecte du **num_veh**), Q2.2 attrape les véhicules fantômes (conducteur absent du PV).

Commentaires. Les 19 580 violations ne sont **pas** des erreurs de qualité à supprimer : elles portent une information réelle (un véhicule a été impliqué dans l'accident, mais son conducteur n'a pas pu être identifié). Cette interprétation est confirmée à la fois par l'inspection manuelle du premier cas et par la stationnarité temporelle du taux. Décision pour la suite : ces véhicules restent comptabilisés dans toutes les analyses centrées sur les véhicules (Q3.1 sur les attributs énumérés **vehicules**, Q4.1 sur les valeurs absentes véhicules, Q6 si répartition par catégorie de véhicule), mais ils n'apparaîtront évidemment pas dans les analyses centrées sur les usagers (gravité, âge, sexe, sécurité). À noter que ce ratio (~1,5 % de véhicules sans conducteur identifié) est lui-même un indicateur intéressant pour la conclusion : la base BAAC capture ainsi indirectement le taux de délit de fuite.

Question 2.3 (facultative) — Lieu → Accident

Rappel de la question. Vérifier par programme que chaque lieu fait référence à un accident présent dans les caractéristiques. Si des données incohérentes existent, elles doivent être toutes listées dans un fichier dédié, analysées et commentées.

Travail avec l'agent — Claude Code

- **Contexte donné à l'agent** : même logique d'intégrité référentielle que Q2.1/Q2.2 mais sur la clé Num_Acc seule — chaque ligne de `lieux_{annee}.csv` doit pointer vers un accident effectivement présent dans `caracteristiques_{annee}.csv`.
- **Sortie attendue** : `results/reponse23.txt` (résumé annuel + analyse) et `results/reponse23_violations.csv` (annee, Num_Acc).
- **Choix d'implémentation** : structure miroir de `question22.py` — pour chaque année on charge l'ensemble des Num_Acc du fichier `caracteristiques_{annee}.csv` dans un `set`, puis on parcourt `lieux_{annee}.csv` ligne à ligne en testant l'appartenance. La clé étant ici un seul champ (et non un couple), le `set` est plus léger (~60-90 k clés/année).
- **Itération mineure** : j'ai ajouté le même sous-décompte « dont Num_Acc vide » que dans Q2.1/Q2.2 pour distinguer une clé manquante d'une vraie référence cassée — il reste à 0, ce qui confirme que tous les Num_Acc côté lieux sont bien renseignés.

Fichier Python : `python/question23.py`.

Vérification (pandas) — `python/verification23.py` → `results/verification23.txt` :

```
>>> f = lieux.merge(caracteristiques, on="Num_Acc",
... how="left", indicator=True)
>>> recap # lieux orphelins par annee
```

```
annee lieux accidents violations
2005 87026 87026 0
2006 82993 82993 0
2007 83850 83850 0
2008 76767 76767 0
2009 74409 74409 0
2010 69379 69379 0
2011 66974 66974 0
2012 62250 62250 0
2013 58397 58397 0
2014 59854 59854 0
2015 58654 58654 0
```

```
# conforme : 0 lieu orphelin (identique)
```

Résultat : `results/reponse23.txt` + `results/reponse23_violations.csv`. Sur **780 553 lieux** cumulés sur 2005-2015, **0 violation** (0,0000 %) : chaque lieu référence un accident existant. Fait notable révélé par le récapitulatif : pour **chaque** année, le nombre de lieux est **strictement égal** au nombre d'accidents distincts (87 026 = 87 026 en 2005, ..., 58 654 = 58 654 en 2015). Le fichier `reponse23_violations.csv` ne contient que son en-tête.

Vérifications.

- *Égalité lieux = accidents* : l'identité parfaite entre `nb_lieux` et `nb_accidents` chaque année est un signal fort — non seulement chaque lieu pointe vers un accident existant (le contrôle demandé), mais il y a **exactement un lieu par accident**. Cela préfigure le résultat de Q2.4 (relation 1-à-1 accident → lieu) et sera confirmé dans ce sens-là.

- *Cohérence des totaux* : la somme des `nb_lieux` (780 553) coïncide avec le cumul `lieux` calculé en Q1.2 — toutes les lignes sont bien balayées.
- *Sous-décompte `Num_Acc` vide* : 0 cas, donc aucune violation « factice » dûe à une clé manquante — le résultat de 0 violation est un vrai 0, pas un artefact de clés vides qui se seraient retrouvées dans le set.
- *Fichier exhaustif* : `wc -l reponse23_violations.csv` = 1 (en-tête seul), cohérent avec 0 violation.

Commentaires. Le résultat est celui attendu par le modèle de données BAAC : le fichier `lieux` décrit la voirie de l'accident à raison d'une ligne par `Num_Acc`, en relation 1-à-1 avec `caracteristiques`. Contrairement à Q2.2 (où les véhicules « fantômes » portaient une information réelle de fuite), il n'y a ici aucune anomalie à interpréter : l'intégrité est parfaite et aucune donnée n'est à exclure pour la suite. L'égalité observée `lieux` = `accidents` chaque année est l'apport principal de cette question — elle garantit aux analyses spatiales ultérieures (Q6 cartographie, Q3 attributs de voirie) qu'une jointure `caracteristiques` ⋈ `lieux` sur `Num_Acc` ne créera ni perte ni duplication de lignes.

Question 2.4 (*facultative*) — Accident → un seul Lieu

Rappel de la question. Vérifier par programme que chaque accident décrit dans les caractéristiques se déroule bien sur un et un seul lieu. Si des données incohérentes existent, elles doivent être toutes listées dans un fichier dédié, analysées et commentées.

Travail avec l'agent — Claude Code

- **Contexte donné à l'agent** : c'est le pendant « cardinalité » de Q2.3. Q2.3 vérifiait le sens lieux → caractéristiques (aucun lieu orphelin) ; ici on vérifie le sens inverse **et le nombre** de lieux par accident — chaque Num_Acc des caractéristiques doit posséder exactement une ligne dans lieux_{annee}.csv.
- **Sortie attendue** : results/reponse24.txt (résumé annuel + analyse) et results/reponse24_violations.csv (Num_Acc, nb_lieux, annee).
- **Choix d'implémentation** : plutôt qu'un simple set comme en Q2.3, on construit pour chaque année un collections.Counter qui compte le nombre d'occurrences de chaque Num_Acc dans lieux_{annee}.csv, puis on parcourt caracteristiques_{annee}.csv en lisant ce compteur. La distinction des deux types d'anomalie a guidé la conception : on sépare explicitement nb_lieux == 0 (accident **sans lieu**) de nb_lieux >= 2 (accident sur **plusieurs lieux**), car ils traduisent des défauts de saisie différents (ligne manquante vs doublon de clé). Le Counter renvoyant 0 pour une clé absente, le cas « sans lieu » est détecté naturellement sans traitement particulier.

Fichier Python : python/question24.py.

Vérification (pandas) — python/verification24.py → results/verification24.txt :

```
>>> nb = carac["Num_Acc"].map(lieux.groupby("Num_Acc").size())
>>> recap # accidents sans lieu / sur plusieurs lieux

annee accidents sans_lieu multi_lieux violations
2005 87026 0 0 0
2006 82993 0 0 0
2007 83850 0 0 0
2008 76767 0 0 0
2009 74409 0 0 0
2010 69379 0 0 0
2011 66974 0 0 0
2012 62250 0 0 0
2013 58397 0 0 0
2014 59854 0 0 0
2015 58654 0 0 0

# conforme : 0 violation : 1 lieu par accident (identique)
```

Résultat : results/reponse24.txt + results/reponse24_violations.csv. Sur **780 553 accidents** cumulés sur 2005-2015, **0 violation** (0,0000 %) : chaque accident se déroule sur un et un seul lieu — 0 accident sans lieu, 0 accident sur plusieurs lieux, et ce pour **chacune** des 11 années. Le fichier reponse24_violations.csv ne contient que son en-tête.

Vérifications.

- *Bijection accident ↔ lieu établie* : combiné à Q2.3 (tout lieu pointe vers un accident existant), ce résultat ferme les deux sens de la relation. Q2.3 montrait déjà l'égalité nb_lieux == nb_accidents chaque année ; Q2.4 prouve que cette égalité globale n'est pas une compensation entre accidents sans lieu et accidents multi-lieux, mais bien une

correspondance terme à terme (1-à-1).

- *Cohérence des totaux* : le cumul de 780 553 accidents balayés coïncide exactement avec les 780 553 lieux de Q2.3 — les deux fichiers ont le même nombre de lignes, confirmation directe de la bijection.
- *Sous-décomptes séparés* : `dont sans lieu = 0` et `dont multi-lieux = 0` chaque année ; aucune des deux familles d'anomalie n'apparaît.
- *Fichier exhaustif* : `wc -l reponse24_violations.csv = 1` (en-tête seul), cohérent avec 0 violation.

Commentaires. Le résultat confirme le modèle de données BAAC : le fichier `lieux` décrit la voirie de l'accident à raison d'une ligne et une seule par `Num_Acc`. Réuni à Q2.3, ce contrôle établit formellement la **bijection** entre `caracteristiques` et `lieux`. Aucune donnée incohérente n'est à lister ni à exclure pour la suite. L'apport pratique est une garantie forte pour les questions ultérieures : une jointure `caracteristiques` $\bowtie$ `lieux` sur `Num_Acc` ne provoquera **ni perte ni duplication** de lignes — les analyses spatiales (Q6 cartographie) et les analyses d'attributs de voirie (Q3) peuvent fusionner les deux fichiers sans précaution particulière sur la cardinalité.

2.2. Domaines énumérés

Question 3.1 — Répartition % des attributs énumérés (usagers)

Rappel de la question. Faire un programme qui détaille la répartition en % des valeurs pour tous les attributs énumérés du fichier des usagers, et commenter le résultat. Les attributs énumérés concernés sont `catu`, `grav`, `sexe`, `trajet`, `secu`, `locp`, `actp`, `etatp`.

Travail avec l'agent — Claude Code

- **Contexte donné à l'agent** : entrée dans `data/enriched/` (sortie de Q1.3), répartition cumulée sur toute la période 2005-2015 (Groupe 1), dénominateur du pourcentage = nombre total d'usagers (chaque ligne fournit exactement une valeur par attribut), affichage trié par % décroissant, code en français.
- **Choix d'implémentation** : un `Counter` par attribut alimenté en un seul parcours `csv.reader` ligne à ligne (pas de `pandas` — un comptage à la lib standard reste plus lisible pour la maintenance).
- **Itération principale** : une subtilité a été identifiée avant rédaction en inspectant l'en-tête et un échantillon des fichiers enrichis (`head -3 usagers_2005.csv`). Sept des huit attributs (`catu`, `grav`, `sexe`, `trajet`, `locp`, `actp`, `etatp`) ont déjà été enrichis en Q1.3 et apparaissent donc sous la forme `code - libellé` (ex. 1 - Conducteur); mais `secu` n'a pas été enrichi car c'est un champ à 2 caractères (1er = équipement de sécurité, 2e = utilisation) qui ne se décode pas par un simple dictionnaire à une clé. L'agent l'a donc traité comme une chaîne brute ("11", "12", "21"...), conformément au piège signalé par le sujet : ne pas le lire comme un entier — "01" ≠ "1") et lui a adjoint, au seul moment de l'affichage, un décodage lisible via deux tables (`SECU_EQUIPEMENT`, `SECU_USAGE`).
- **Décision validée** : ne pas masquer les valeurs absentes ("", ".", -1) ni les codes hors documentation — ils sont comptés et affichés explicitement pour pouvoir être commentés (et réutilisés en Q4).

Fichier Python : `python/question31.py`.

Vérification (`pandas`) — `python/verification31.py` → `results/verification31.txt` :

```
>>> recap = usagers[attr].value_counts() # par modalite
>>> recap.sum() == total_usagers # bon denominateur ?
>>> recap # somme effectifs / attribut

attribut modalites somme_eff = total
  catu 4 1742583 oui
  grav 4 1742583 oui
  sexe 2 1742583 oui
trajet 8 1742583 oui
  locp 10 1742583 oui
  actp 9 1742583 oui
  etatp 5 1742583 oui

# conforme : effectifs identiques a reponse31.txt, denominateur 1742583
```

Résultat : `results/reponse31.txt` (un tableau par attribut, trié par % décroissant, sur 1 742 583 usagers cumulés). Principaux constats :

- **catu** : 74,52 % conducteurs, 17,04 % passagers, 8,27 % piétons, 0,17 % piétons en roller/trottinette (4 valeurs documentées, aucune absente).
- **grav** : 40,76 % indemnes, 35,59 % blessés légers, 20,96 % blessés hospitalisés, 2,69 % tués (46 934 personnes).
- **sexe** : 66,89 % hommes, 33,11 % femmes (2 valeurs, aucune absente).

- **trajet** : 36,65 % promenade-loisirs, 29,28 % non renseigné (0), 12,91 % domicile-travail ; seules 359 cellules (0,02 %) sont réellement vides.
- **secu** : 26 valeurs distinctes. 55,28 % « Ceinture portée » (11), 18,29 % « Casque porté » (21), 8,00 % « Ceinture – non déterminable » (13). Le décodage 2 caractères fonctionne : la majorité des codes correspond à la documentation. Restent ~4 % de 00 et quelques codes mono-caractère (1, 2, 3) ou en 0 (10, 20, 90...) **hors documentation**, plus 1,96 % de cellules vides.
- **locp** / **actp** / **etatp** (attributs *piéton*) : très majoritairement 0 (« sans objet » : 92,39 % / 91,90 % / 92,00 %), ce qui est cohérent puisque seuls 8,27 % des usagers sont des piétons. Les valeurs non nulles sont toutes documentées (passage piéton, traversant, seul/accompagné...), avec < 0,15 % d'absents par attribut.

Vérifications réalisées + analyse.

- *Conservation du total* : pour chaque attribut, la somme des effectifs affichés vaut bien 1 742 583 (chaque usager compte une fois), ce qui valide le dénominateur commun et garantit que la somme des pourcentages fait 100 %.
- *Conformité à la documentation* : pour les 7 attributs enrichis, toutes les valeurs portent un libellé **code** – libellé issu du mapping Q1.3 ou sont des marqueurs d'absence explicites — aucune valeur enrichie « orpheline ». Pour **secu**, le décodage 2 caractères couvre l'écrasante majorité des effectifs ; les codes restants (00, mono-caractères, codes en 0) sont signalés (**code hors documentation**) plutôt que faussement libellés.
- *Cohérence transversale* : la quasi-absence de valeurs **locp/actp/etatp** non nulles (~8 %) recoupe exactement la part de piétons donnée par **catu** (8,27 %) — les attributs spécifiques aux piétons ne sont renseignés que pour les piétons, signe d'une saisie cohérente.
- *Lecture métier* : la prédominance des conducteurs masculins, le pic « promenade-loisirs », et surtout le port quasi systématique de ceinture/casque chez les usagers indemnes ou légèrement blessés sont conformes aux tendances connues de l'accidentologie française.

Commentaires. Le résultat est exploitable tel quel pour les analyses ultérieures. Deux points appellent une vigilance pour la suite : (1) le champ **secu** mélange des codes non documentés (notamment 00, ~3,9 %) et des codes tronqués à un caractère, à traiter comme « non renseignés » plutôt que comme une vraie modalité ; (2) le marqueur 0 de **trajet**, **locp**, **actp**, **etatp** n'est pas une donnée manquante au sens strict mais un « non renseigné / sans objet » documenté — distinction importante pour Q4 (taux de valeurs absentes) où ces 0 ne devront pas être confondus avec des cellules vides.

2.3. Valeurs absentes

Question 4.1 — Taux de valeurs absentes (véhicules)

Rappel de la question. Faire un programme qui mesure le taux de valeurs absentes pour chaque champ du fichier des véhicules et interpréter les résultats.

Travail avec l'agent — Claude Code

- **Contexte donné à l'agent** : entrée dans `data/enriched/`, cumul sur toute la période 2005-2015 (Groupe 1), dénominateur du taux = nombre total de véhicules, code en français, sortie `results/reponse41.txt` (tableau colonne, `nb_manquants`, `taux_%` + interprétation). Le point central de la discussion a porté sur la **définition d'une valeur absente**. Conformément à la convention BAAC (`schema_baac.md` §2), on retient comme marqueurs : cellule vide "", point '.', '-1', et le zéro sous ses différentes largeurs '0'/'00'/'000'. Une subtilité a été traitée avant rédaction en inspectant l'en-tête et un échantillon (`head -3 vehicules_2005.csv`) : cinq colonnes (`catv`, `obs`, `obsm`, `choc`, `manv`) ont été **enrichies** en Q1.3 et apparaissent sous la forme `code - libellé` (ex. 07 - VL seul), alors que `senc`, `occutc`, `num_veh` et `Num_Acc` sont restées brutes. Le test d'absence isole donc le **code** (partie avant « - ») pour les colonnes enrichies, et porte sur la valeur brute sinon.
- **Décision assumée et signalée dans l'énoncé** : le code '0' n'a pas le même sens partout — il est compté mécaniquement comme absent, mais l'interprétation distingue le « non renseigné » du « sans objet » documenté (cf. point de vigilance ci-dessous).
- **Implémentation à la librairie standard** : un simple compteur d'absences par colonne alimenté en un seul parcours `csv.reader` (pas de `pandas`), pour rester lisible à la soutenance.

Fichier Python : `python/question41.py`.

Vérification (pandas) — `python/verification41.py` → `results/verification41.txt` :

```
>>> code = col.str.split(" - ").str[0] # code avant " - "
>>> code.isin(MARQUEURS).sum() # nb manquants
>>> recap # par colonne

colonne nb_manquants taux_%
occutc 1322304 99.31
senc 1296378 97.36
obs 1159679 87.10
obsm 292866 22.00
manv 106486 8.00
choc 88711 6.66
Num_Acc 0 0.00
catv 0 0.00
num_veh 0 0.00

# conforme : nb_manquants identiques a reponse41.txt colonne par colonne
```

Résultat : `results/reponse41.txt` (tableau trié par taux décroissant, sur **1 331 465 véhicules** cumulés). Taux de valeurs absentes par champ :

colonne	nb_manquants	taux_%	qualificatif
occutc	1 322 304	99,31 %	fragile
senc	1 296 378	97,36 %	fragile
obs	1 159 679	87,10 %	fragile
obsm	292 866	22,00 %	fragile
manv	106 486	8,00 %	prudence
choc	88 711	6,66 %	prudence
Num_Acc	0	0,00 %	bien renseigné
catv	0	0,00 %	bien renseigné
num_veh	0	0,00 %	bien renseigné

Seuils d'interprétation appliqués : < 5 % bien renseigné, 5-15 % prudence, > 15 % fragile.

Vérifications réalisées + analyse.

- *Identifiants intacts* : Num_Acc et num_veh (clés de jointure) sont à 0 % d'absence, ce qui est attendu et cohérent avec Q2.1/Q2.2 où ces clés ont permis des jointures sans clé manquante.
- *occutc (99,31 %)* : le marqueur '000' domine, car la quasi-totalité des véhicules ne sont **pas** des transports en commun. Il s'agit d'un « sans objet » massif et non d'un défaut de saisie : la colonne n'est pertinente que pour les bus/cars.
- *obs (87,10 %)* : très majoritairement « 00 - **Sans objet** ». Le taux élevé traduit la **rareté des chocs contre obstacle fixe**, pas une absence de remplissage — le 0 est ici une vraie information.
- *obsm (22,00 %) et choc (6,66 %)* : utilisent aussi le code 0 documenté (« Aucun »), donc leur taux mêle absences réelles et « sans objet », à interpréter avec recul.
- *senc (97,36 %)* : le code '0' n'étant pas documenté (seuls 1 et 2 le sont), il est compté comme absent ; le taux reflète une **saisie irrégulière** du sens de circulation, variable selon les années (point de vigilance signalé par l'énoncé).
- *catv (0 %)* : l'attribut le mieux renseigné, ce qui est rassurant car beaucoup d'analyses (Q3, Q6) en dépendent.

Commentaires. La convention « 0 = absent » est volontairement **stricte** : pour les champs où 0 est documenté (*obs*, *obsm*, *choc*, *occutc*), un taux élevé signale surtout du « sans objet » et non un défaut de remplissage — ces champs restent exploitables à condition de **distinguer 0 des autres valeurs** lors des analyses ultérieures. Le seul champ réellement préoccupant en termes de saisie est *senc*, dont l'absence n'est pas un « sans objet » légitime. Ce résultat prolonge directement la vigilance soulevée en Q3.1 : le marqueur 0 ne doit jamais être confondu avec une cellule vide, et le sens de l'absence dépend de la sémantique propre à chaque attribut.

2.4. Cohérence inter-attributs

Question 5.1 — Incohérence `lum` × `hrmn`

Rappel de la question. Compter les accidents pour lesquels la luminosité déclarée (`lum`) contredit l'heure de l'accident (`hrmn`), afficher les 10 premiers cas, générer le rapport complet dans un fichier, puis commenter.

Travail avec l'agent — Claude Code

Contexte donné : lecture de `data/enriched/`, cumul sur 2005-2015 (Groupe 1), code en français, deux sorties (`results/reponse51.txt` pour le résumé + 10 cas + commentaires, et `results/reponse51_complet.csv` pour le rapport exhaustif). Deux points ont été tranchés avant rédaction par **inspection des données** (et non par hypothèse) :

- *Format de `hrmn`* : un `python3` jetable sur les 11 fichiers a montré que `hrmn` est un entier HHMM **non zéro-paddé** — les longueurs valent 1 à 4 chiffres ("`230`" = 02 h 30, "`800`" = 08 h 00, "`1900`" = 19 h 00). La suggestion initiale `int(hrmn[:2])` aurait donné une heure fausse pour tout accident avant 10 h ("`800`"[:2] = 80). On a donc retenu `int(hrmn) // 100`, correct pour toutes les largeurs. La même inspection confirme **0 cellule vide ou non numérique** sur les 780 553 accidents ; la garde sur les `hrmn` invalides est conservée par robustesse mais ne filtre rien ici.
- *Définition de l'incohérence* : on ne signale que les cas **francs**, avec des bornes **dépendant de la saison** (déduite du mois) pour neutraliser les faux positifs liés au lever/coucher du soleil très variables. Pour chaque saison, une plage « plein jour plausible » et une plage « cœur de journée » (où il fait forcément jour) : hiver 8–17 h / 10–16 h, printemps 7–20 h / 9–17 h, été 6–21 h / 8–18 h, automne 7–20 h / 9–17 h. Plein jour déclaré hors de la plage de jour de sa saison → incohérent ; nuit (3/4/5) déclarée dans le cœur de journée → incohérent. Le crépuscule/aube (2 - ...) est **exclu** du test, car c'est par nature une luminosité d'heure intermédiaire qu'aucune borne ne peut qualifier sans connaître la date exacte. Implémentation à la librairie standard (`csv.reader`, un seul parcours par année), sans `pandas`.

Fichier Python : `python/question51.py`.

Vérification (`pandas`) — `python/verification51.py` → `results/verification51.txt` :

```
>>> # bornes par saison ; heure = int(hrmn) // 100
>>> recap = complet["motif"].value_counts()
>>> recap

              motif n
Plein jour déclaré mais heure nocturne 6742
Nuit déclarée mais heure en pleine journée 5292

# conforme : 12034 incoherences (identique au recompute pandas saisonnier)
```

Résultat : `results/reponse51.txt` (résumé + 10 premiers cas + commentaires) et `results/reponse51_complet.csv` (12 034 lignes, colonnes `annee`, `Num_Acc`, `hrmn`, `heure`, `saison`, `lum`, `motif`). Bilan sur **780 553** accidents datés :

Indicateur	Valeur
Accidents testés (<code>hrmn</code> renseignée)	780 553
Total incohérences	12 034 (1,542 %)
dont « plein jour déclaré la nuit »	6 742
dont « nuit déclarée en pleine journée »	5 292

Extrait des 10 premiers cas : 200500000322 à 22 h 15 codé « 1 - Plein jour » (erreur nette), 200500000165 à 13 h 30 codé « 3 - Nuit sans éclairage public », etc.

Vérifications réalisées + analyse.

- *Format horaire validé* : extraction par `int(hrmn)//100` vérifiée sur des cas limites ("645"→6, "2215"→22) ; comptage des lignes du CSV (`wc -l` = 12 035 = 12 034 + en-tête) cohérent avec le total affiché.
- *Cohérence globalement très bonne* : 1,54 % d'incohérences seulement — les deux champs sont renseignés de façon largement concordante par les forces de l'ordre.
- *Asymétrie modérée* : 6 742 cas « plein jour la nuit » contre 5 292 « nuit en plein jour ». Les bornes **saisonnnières** ont précisément pour but d'éliminer les faux positifs des heures charnières (un accident « plein jour » à 6 h–7 h ou à 20 h est jugé cohérent en été, incohérent en hiver) : les cas restants sont donc **francs** dans les deux sens, d'où un écart bien plus équilibré qu'avec des bornes fixes.

Commentaires. Le taux de 1,54 % mêle deux populations qu'on ne peut pas séparer à partir des seuls champs `lum` et `hrmn` : de **vraies erreurs de saisie** (luminosité ou heure mal reportée — ex. 22 h en « plein jour ») et des **situations réelles mais rares** (tunnel, brouillard très dense, éclipse). Le choix de bornes **saisonnnières** prudentes privilégie la précision (peu de faux positifs, même aux heures charnières) au prix d'un rappel non exhaustif. Le rapport complet `reponse51_complet.csv` permet d'inspecter chaque cas individuellement, et prolonge la démarche de contrôle qualité engagée en Q4 et Q5 : les incohérences inter-attributs restent marginales mais bien réelles.

Question 5.2 — Autre incohérence inter-attributs (`obsbm` × `choc`)

Rappel de la question. Identifier une situation différente (autre que `lum` × `hrmn`) où il peut y avoir des incohérences inter-attributs, compter les lignes en incohérence, afficher les 10 premières et générer le rapport complet dans un fichier, puis commenter.

Travail avec l'agent — Claude Code

Situation retenue : la contradiction entre l'**obstacle mobile heurté** (`obsbm`) et le **point de choc initial** (`choc`), deux attributs de la table *vehicules*. Si `obsbm` désigne un obstacle mobile effectivement heurté mais que `choc` = « 0 - Aucun », il y a contradiction : on ne peut pas heurter quelque chose sans point d'impact sur le véhicule. Deux points ont été tranchés par **inspection des données** avant rédaction :

- *Valeurs « aucun » et cellules vides* : un `python3` jetable sur les 11 fichiers *vehicules* a listé toutes les modalités. La valeur « aucun » se repère au préfixe « 0 - » (format enrichi). Les cellules vides (551 `obsbm`, 218 `choc` sur ~1,3 M de lignes) sont **ignorées** : champ non renseigné → ligne non testable. Une ligne n'est comptée comme « testée » que si `obsbm` et `choc` sont tous deux renseignés.
- *Asymétrie de la règle* : on ne signale que le cas « obstacle mobile heurté **sans** point de choc ». Le cas inverse (un `choc` renseigné mais `obsbm` = « 0 - Aucun ») n'est **pas** une incohérence, car le véhicule a pu heurter un obstacle **fixe** (mur, glissière, arbre...), codé dans `obs` et non dans `obsbm`. Implémentation à la librairie standard (`csv.reader`, un seul parcours par année), sans `pandas`, dans la continuité de Q5.1.

Fichier Python : `python/question52.py`.

Vérification (pandas) — `python/verification52.py` → `results/verification52.txt` :

```
>>> obstacle = obsbm.str[0].isin(list("124569"))
>>> incoh = vehicules[obstacle & choc.str.startswith("0 - ")]
>>> recap # incoherences par annee

annee incoherences
2005 3233
2006 3217
2007 3717
2008 4214
2009 3888
2010 3592
2011 3040
2012 3127
2013 3621
2014 3515
2015 4484

# conforme : 39648 incoherences obsbm x choc (identique)
```

Résultat : `results/reponse52.txt` (résumé + 10 premiers cas + commentaires) et `results/reponse52_comp` (39 648 lignes, colonnes `annee`, `Num_Acc`, `num_veh`, `obsbm`, `choc`). Bilan sur **1 330 827** lignes véhicules testées :

Indicateur	Valeur
Lignes véhicules testées (obsm et choc renseignés)	1 330 827
Total incohérences	39 648 (2,979 %)
dont obstacle « 2 - Véhicule »	27 521
dont obstacle « 1 - Piéton »	10 361
dont obstacle « 9 - Autre »	1 478
dont animaux / rail (5, 6, 4)	288

Extrait des 10 premiers cas : 200500000004 (véhicule B02) déclare « 2 - Véhicule » heurté mais choc = « 0 - Aucun » ; 200500000696 (A01) déclare « 1 - Piéton » heurté sans point de choc, etc.

Vérifications réalisées + analyse.

- *Comptage cohérent* : `wc -l reponse52_complet.csv` = 39 649 = 39 648 + en-tête, conforme au total affiché.
- *Plausibilité de la règle* : les obstacles dominants dans les incohérences (« Véhicule » 69 %, « Piéton » 26 %) sont aussi les plus fréquents dans obsm, ce qui est attendu — il n'y a pas de biais vers un type d'obstacle rare.
- *Taux faible (2,98 %)* : la grande majorité des couples (obsm, choc) sont concordants. Le remplissage de ces deux champs par les forces de l'ordre est globalement fiable.

Commentaires. Les 39 648 incohérences s'expliquent presque toujours par un **oubli de saisie sur le champ choc** (laissé à sa valeur par défaut « 0 - Aucun ») alors que obsm a, lui, été correctement renseigné. La règle a été délibérément rendue **asymétrique** pour éviter les faux positifs : un point de choc sans obstacle mobile est parfaitement légitime (collision contre un obstacle fixe). Le rapport complet `reponse52_complet.csv` identifie chaque ligne fautive par son Num_Acc et son num_veh, ce qui permet de remonter au véhicule précis et de croiser avec les autres tables. La démarche prolonge le contrôle qualité inter-attributs amorcé en Q5.1 sur un autre couple de champs.

Question 5.3 (*facultative*) — Troisième incohérence (`an_nais` × période)

Rappel de la question. Identifier une troisième situation, différente des deux précédentes, où il peut y avoir des incohérences inter-attributs ; compter les lignes en incohérence, afficher les 10 premières et générer le rapport complet dans un fichier, puis commenter.

Travail avec l'agent — Claude Code

Situation retenue : une **année de naissance impossible** au regard de la **période d'étude**. Dans la table *usagers*, l'attribut `an_nais` est confronté à la fenêtre 2005-2015 (Groupe 1), implicitement portée par l'année du fichier. Une ligne est jugée incohérente si `an_nais` est renseignée mais désigne une naissance matériellement impossible :

- `an_nais > 2015` → l'utilisateur naîtrait *après* l'accident le plus tardif de la fenêtre (impossible d'être victime avant d'être né) ;
- `an_nais < 1900` → l'utilisateur aurait plus de 105 ans au moment des faits (valeur quasi certainement erronée).

Deux points ont été tranchés par **inspection des données** avant rédaction :

- *Valeurs manquantes* : un `python3` jetable sur les fichiers *usagers* a confirmé que `an_nais` est soit vide, soit une année. Les codes de valeur manquante (cellule vide, « 0 », « 00 », « . ») et toute valeur non numérique sont **ignorés** : non-réponse → ligne non testable. Les y inclure gonflerait artificiellement le compte.
- *Bornes volontairement larges* : on ne retient que les cas **francs** — la naissance dans le futur (strictement impossible) et l'âge > 105 ans (extrême). On évite ainsi de qualifier d'« incohérent » un centenaire réel. Implémentation à la librairie standard (`csv.reader`, un seul parcours par année), sans `pandas`, dans la continuité de Q5.1 et Q5.2.

Fichier Python : `python/question53.py`.

Vérification (`pandas`) — `python/verification53.py` → `results/verification53.txt` :

```
>>> an = usagers["an_nais"].astype(int)
>>> incoh = usagers[(an > 2015) | (an < 1900)]
>>> recap # extrait (41 cas au total)

annee Num_Acc num_veh an_nais motif
2005 200500003366 B01 1898 Naissance trop ancienne, > 105 ans (an_nais < 1900)
2005 200500006218 A01 1898 Naissance trop ancienne, > 105 ans (an_nais < 1900)
2005 200500007379 A01 1898 Naissance trop ancienne, > 105 ans (an_nais < 1900)
2005 200500010227 B01 1898 Naissance trop ancienne, > 105 ans (an_nais < 1900)
2005 200500015680 B01 1898 Naissance trop ancienne, > 105 ans (an_nais < 1900)
2005 200500018076 B01 1898 Naissance trop ancienne, > 105 ans (an_nais < 1900)

# conforme : 41 annees de naissance impossibles (identique)
```

Résultat : `results/reponse53.txt` (résumé + 10 premiers cas + commentaires) et `results/reponse53_comp` (41 lignes, colonnes `annee`, `Num_Acc`, `num_veh`, `an_nais`, `motif`). Bilan sur **1 740 264** lignes usagers testées :

Indicateur	Valeur
Lignes usagers testées (<code>an_nais</code> renseignée)	1 740 264
Total incohérences	41 (0,002 %)
dont naissance postérieure à 2015	0
dont naissance antérieure à 1900	41

Extrait des 10 premiers cas : 200500003366 (véhicule B01) avec `an_nais` = 1898, 200500034919 (B01) avec `an_nais` = 1897, etc. — tous des millésimes antérieurs à 1900.

Vérifications réalisées + analyse.

- *Comptage cohérent* : `wc -l reponse53_complet.csv` = 42 = 41 + en-tête, conforme au total affiché.
- *Répartition* : les 41 cas sont **tous** des naissances antérieures à 1900 ; aucune naissance postérieure à 2015 n'apparaît dans la fenêtre, ce qui est cohérent avec un enregistrement administratif où les âges aberrants viennent surtout de relevés anciens.
- *Taux infime (0,002 %)* : la qualité de remplissage de `an_nais` est excellente. Sur 1,74 M d'utilisateurs, à peine 41 valeurs sont matériellement impossibles.

Commentaires. Ces 41 incohérences relèvent d'**erreurs de saisie ou de relevé** (inversion de chiffres, millésime fantaisiste type 1898). Le contrôle complète utilement Q5.1 (`lum` × `hrmn`, table *caractéristiques*) et Q5.2 (`obsn` × `choc`, table *vehicules*) en portant cette fois sur la table *usagers* et en confrontant un attribut non pas à un autre champ de la même ligne, mais à une **contrainte externe** (la période d'étude) — ce qui élargit la notion d'incohérence inter-attributs. Le rapport complet `reponse53_complet.csv` identifie chaque ligne fautive par son `Num_Acc` et son `num_veh`, permettant de remonter à l'utilisateur précis.

3. Requêtage et visualisation

Question 6.1 — Accidents par tranche horaire d'1h

Rappel de la question. Afficher le nombre d'accidents par tranche horaire d'une heure.

Travail avec l'agent — Claude Code

La donnée source est l'heure de survenue `hrmn` de la table *caracteristiques* (`data/enriched/`), cumulée sur toute la fenêtre 2005-2015 (Groupe 1) et restreinte à la France métropolitaine via `normaliser_dep` (exclusion des DOM et des départements non métropolitains). Deux points ont été tranchés avant rédaction :

- *Extraction de l'heure* : `hrmn` est au format HHMM **sans zéro de tête** — « 1900 » vaut 19h00, mais « 300 » vaut 3h00 et « 30 » vaut 0h30. Découper les deux premiers caractères (`int(hrmn[:2])`) serait donc **faux** pour toute valeur à moins de 4 chiffres (« 300 » donnerait 30h). L'heure est obtenue de façon fiable par division entière : `int(hrmn) // 100`.
- *Valeurs manquantes* : les `hrmn` vides ou non numériques sont **ignorées** (heure non testable). En pratique le décompte affiche **0 valeur ignorée** : `hrmn` est intégralement renseignée sur la période.

Le rendu matplotlib est configuré en mode hors écran (`matplotlib.use("Agg")`) pour écrire directement le PNG sans interface graphique. Implémentation à la librairie standard (`csv.reader`, un seul parcours par année), dans la continuité des questions précédentes. C'est la première question imposant un graphique : la dépendance `matplotlib` a été ajoutée au projet (`uv add matplotlib`).

Fichier Python : `python/question61.py`.

Vérification (pandas) — `python/verification61.py` → `results/verification61.txt` :

```
>>> h = caracteristiques.loc[metropole, "hrmn"].astype(int) // 100
>>> recap = h.value_counts().sort_index()
>>> recap # extrait : 8 premieres heures

heure accidents
0 12819
1 10616
2 8926
3 7211
4 7973
5 10640
6 13857
7 32476

# conforme : 24 tranches identiques a question61.py (total 757263)
```

Résultat : `results/reponse61.png` — graphique en barres verticales bleues (axe x = heure 0-23, axe y = nombre d'accidents), titre « Accidents par tranche horaire (2005-2015) », grille horizontale, taille 12×6 pouces. Bilan sur **757 263** accidents métropolitains :

Indicateur	Valeur
Total accidents (métropole, 2005-2015)	757 263
Valeurs <code>hrmn</code> ignorées	0
Heure de pointe	17h (66 958 accidents)
Second pic	18h (65 343) puis 8h (44 033)
Heure la plus calme	3h (7 211 accidents)
Cumul nuit profonde (0h-5h)	58 185 (≈ 7,7 %)

Vérifications réalisées + analyse.

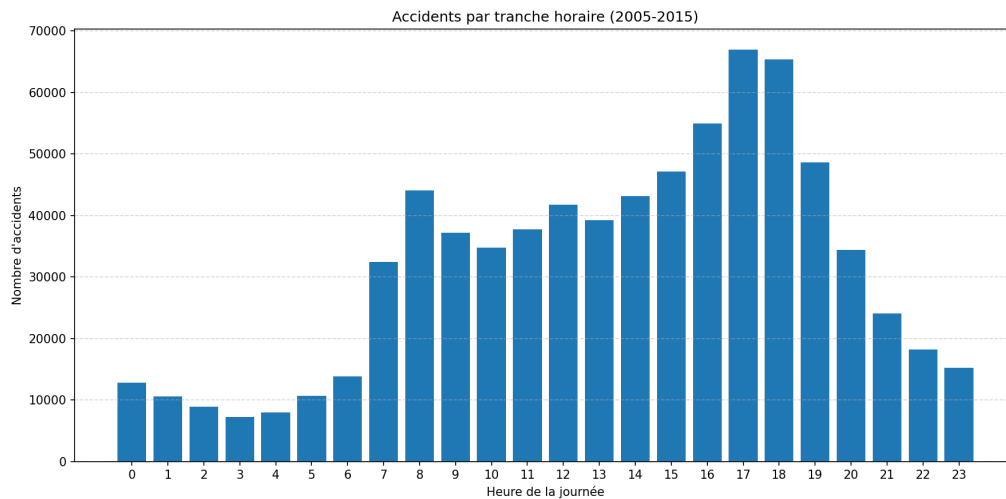


FIGURE 1 – Q6.1 — Nombre d'accidents par tranche horaire d'une heure (métropole, 2005-2015)

- *Conservation du total* : la somme des 24 tranches (757 263) coïncide exactement avec le total des accidents métropolitains cumulés sur la période, et aucune valeur n'est perdue (0 ignorée). L'histogramme couvre bien les 24 heures.
- *Plausibilité de la forme* : la distribution est **bimodale** et épouse les déplacements domicile-travail — un pic le matin (8h, 44 033) et un pic plus marqué le soir (17h-18h, $\approx 66\,000$), un creux la nuit (minimum à 3h, 7 211). Ce profil est conforme à ce qu'on attend du trafic routier et constitue un test de cohérence du traitement.
- *Robustesse de l'extraction* : un contrôle sur des valeurs courtes (« 300 », « 30 ») confirme que `int(hrmn) // 100` les classe correctement (3h et 0h), là où un découpage de chaîne aurait produit des heures aberrantes.

Commentaires. Le graphique met en évidence que les accidents ne sont pas répartis uniformément dans la journée : ils suivent l'intensité du trafic. Le pic du soir (17h-18h) dépasse celui du matin, ce qui s'explique classiquement par un trafic de retour plus étalé, une fatigue de fin de journée et une luminosité déclinante en partie de l'année. Les heures nocturnes (0h-5h) concentrent peu d'accidents en valeur absolue ($\approx 7,7\%$ du total) — mais ce volume faible ne préjuge pas de leur gravité, qui sera examinée dans les questions suivantes.

Question 6.2 — Tués cumulés par tranche d'âge et sexe

Rappel de la question. Fournir un graphique présentant le nombre de tués cumulé sur la période du projet par tranche d'âge (de 5 années) et par sexe.

Travail avec l'agent — Claude Code

La donnée source est la table *usagers* (*data/enriched/*), cumulée sur 2005-2015 (Groupe 1). Trois points ont été tranchés avant rédaction :

- *Définition d'un tué* : on filtre `grav == "2 - Tué"` (le label exact des fichiers enrichis). Le sexe est lu sur `sexe` (« 1 - Masculin » / « 2 - Féminin »).
- *Calcul de l'âge* : `age = annee_fichier - an_nais`. L'année est lue dans le nom du fichier (*usagers_2005.csv* → 2005), qui est ici un **millésime complet à 4 chiffres** ; `an_nais` est lui aussi un millésime complet — aucune reconstruction du siècle n'est donc nécessaire. Une `an_nais` vide, non numérique, égale à « 0 », ou donnant un âge hors de l'intervalle plausible `[0, 120]` est ignorée (96 cas sur la période, soit 0,2 % des tués).
- *Filtrage métropole* : la table *usagers* ne porte pas le département. Pour chaque année on récupère l'ensemble des `Num_Acc` métropolitains via `charger_num_acc_metropole` (jointure sur la table *caractéristiques* + `normaliser_dep`, qui exclut DOM et départements non métropolitains), et on ne conserve que les usagers de ces accidents — même logique de jointure que Q2.

Les tranches sont de 5 ans : 18 tranches 0-4 ... 85-89 plus une tranche ouverte 90+ (les âges ≥ 90 y sont regroupés), soit 19 tranches. Le graphique est en **barres groupées** : deux barres par tranche (hommes en bleu, femmes en rouge), légende et grille horizontale. Implémentation à la librairie standard (`csv.reader`, un seul parcours par année) avec rendu matplotlib hors écran (`matplotlib.use("Agg")`), dans la continuité de Q6.1.

Fichier Python : `python/question62.py`.

Vérification (pandas) — `python/verification62.py` → `results/verification62.txt` :

```
>>> tues = usagers[(usagers.grav == "2 - Tué") & metropole]
>>> recap = tues.groupby([tranche_age, "sexe"]).size()
>>> recap # extrait : 8 premieres tranches

tranche hommes femmes
0-4 223 174
5-9 191 159
10-14 313 190
15-19 3145 860
20-24 5601 1176
25-29 4015 694
30-34 2863 513
35-39 2516 511

# conforme : effectifs identiques a question62.py (H=33921, F=10899)
```

Résultat : `results/reponse62.png` — barres groupées (axe x = tranche d'âge, 2 barres par tranche, axe y = nombre de tués cumulés), titre « Tués cumulés par tranche d'âge et par sexe (2005-2015) », taille 14×7 pouces. Bilan sur **44 820** tués métropolitains :

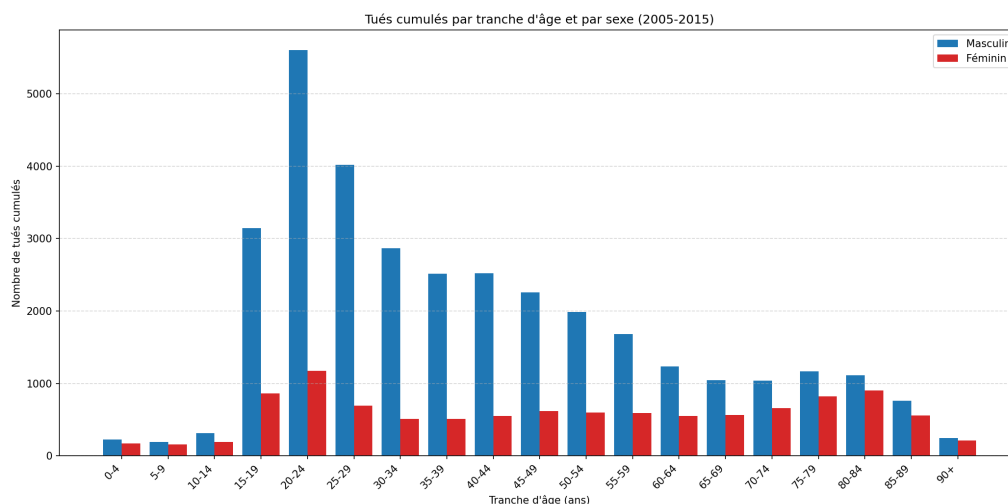


FIGURE 2 – Q6.2 — Tués cumulés par tranche d'âge (5 ans) et par sexe (métropole, 2005-2015)

Indicateur	Valeur
Total tués (métropole, 2005-2015)	44 820
dont hommes	33 921 (75,7 %)
dont femmes	10 899 (24,3 %)
Tranche la plus touchée	20-24 ans (6 777 tués)
Tués au sexe non renseigné	0
Tués à l'âge non exploitable (ignorés)	96

Vérifications réalisées + analyse.

- *Conservation des effectifs* : la somme hommes + femmes des 19 tranches (44 820) coïncide avec le cumul des tués métropolitains affiché année par année, et le sexe est intégralement renseigné chez les tués (0 ignoré). Seuls 96 tués (0,2 %) sont écartés faute d'âge exploitable.
- *Plausibilité de la forme* : la distribution présente un **pic marqué sur les 20-24 ans** (6 777 tués) puis 15-19 et 25-29, conforme à la surmortalité routière bien documentée des jeunes adultes (prise de risque, inexpérience), avant une décroissance régulière avec l'âge.
- *Plausibilité de l'écart hommes/femmes* : les hommes représentent **75,7 %** des tués, un déséquilibre constant sur toutes les tranches et cohérent avec les statistiques nationales de la sécurité routière (exposition au volant et profils de conduite). Un léger resserrement de l'écart apparaît aux âges élevés.

Commentaires. Le graphique croise deux facteurs de risque — l'âge et le sexe — et fait ressortir une population particulièrement exposée : les **hommes jeunes (15-29 ans)**, qui concentrent à eux seuls une part majeure des tués. La tranche 90+, ouverte, reste résiduelle en valeur absolue. Ce résultat éclaire les questions de gravité : au-delà du volume d'accidents (Q6.1), c'est ici la mortalité qui est ventilée, et elle ne suit pas la même structure démographique que l'ensemble des usagers impliqués.

Question 6.3 — Top 10 départements (carte folium)

Rappel de la question. Quels sont les 10 départements avec le plus d'accidents ? L'affichage se fait sur une carte de France en utilisant la bibliothèque **folium**.

Travail avec l'agent — Claude Code

La donnée source est la table *caractéristiques* (`data/enriched/`), cumulée sur 2005-2015 (Groupe 1). Chaque ligne de ce fichier correspond à un accident (Num_Acc unique par fichier-année) : le décompte par département est donc un simple comptage de lignes, sans jointure. Trois points ont été tranchés :

- *Identification du département* : le champ `dep` des fichiers BAAC 2005-2015 est codé sur le modèle `département × 10` (ex. Paris = 750, Nord = 590). On le repasse au code INSEE à 2 chiffres via `normaliser_dep`, qui gère aussi la Corse (201 → 2A, 202 → 2B) et **exclut DOM et codes non métropolitains** (renvoie `None`). Seuls les départements normalisés non `None` sont comptés.
- *Coordonnées* : les cercles sont placés sur le **centroïde** de chaque département, lu dans `CENTROIDES_DEPARTEMENTS` (`common.py`). Le nom complet du département (pour les popups) est porté par un dictionnaire `NOMS_DEPARTEMENTS` local au script — `common.py` n'expose que les codes, on a préféré ne pas le modifier.
- *Lisibilité de la carte* : les **10 premiers** départements sont des **cercles rouges proportionnels** au nombre d'accidents (rayon en pixels normalisé sur le maximum, Paris, entre ~8 et 40 px), avec popup détaillée et tooltip au survol ; **tous les autres** départements sont de **petits marqueurs gris discrets** (rayon 2 px), pour situer le top 10 dans l'ensemble du territoire sans surcharger. Fond de carte `CartoDB positron` (sobre), centrage (46.5, 2.0), zoom 6. Implémentation : un parcours `csv.reader` par année + un `collections.Counter`, puis `folium.CircleMarker`.

Fichier Python : `python/question63.py`.

Vérification (pandas) — `python/verification63.py` → `results/verification63.txt` :

```
>>> recap = (caracteristiques["dep"].map(normaliser_dep)
... .value_counts().head(10))
>>> recap # top 10 departements

rang dep accidents
1 75 81016
2 13 48078
3 93 30779
4 92 28388
5 06 27038
6 94 26668
7 69 23658
8 59 22657
9 33 20813
10 91 16044

# conforme : meme top 10 (departement + effectif) que reponse63.txt
```

Résultat : `results/reponse63.html` (carte interactive) et `results/reponse63.txt` (tableau). Sur **757 263** accidents métropolitains cumulés répartis sur 96 départements, le classement est :

Q6.3 — Accidents cumulés par département (2005-2015)
Top 10 en rouge — aperçu statique de la carte Folium

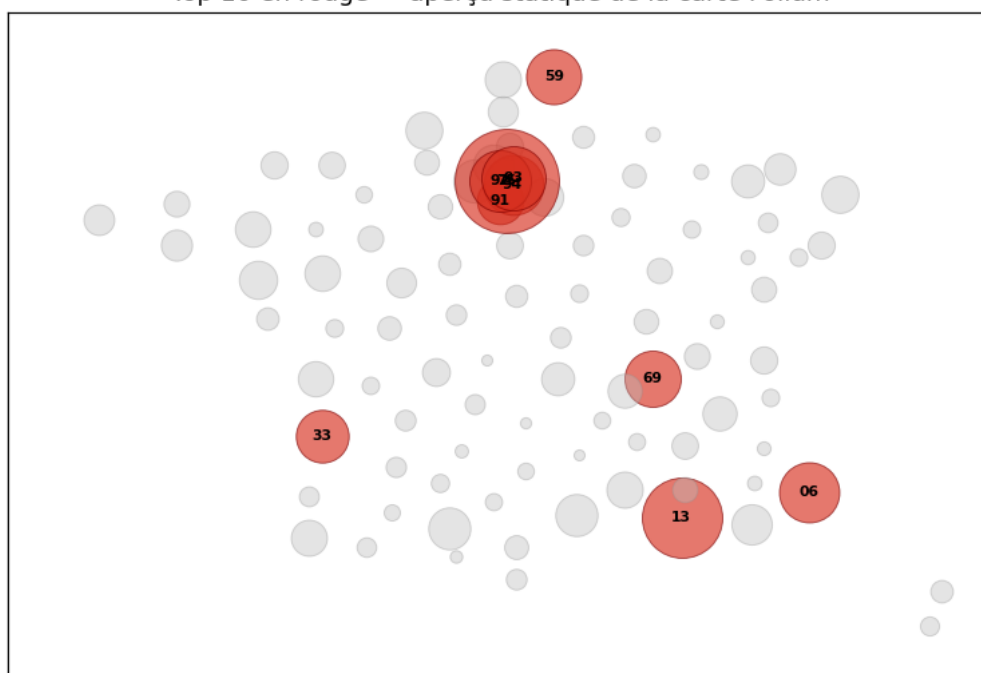


FIGURE 3 – Q6.3 — Aperçu statique de la carte Folium : accidents cumulés par département, top 10 en rouge (métropole, 2005-2015). La carte interactive est dans `results/reponse63.html`.

Rang	Dép.	Nom	Nb accidents
1	75	Paris	81 016
2	13	Bouches-du-Rhône	48 078
3	93	Seine-Saint-Denis	30 779
4	92	Hauts-de-Seine	28 388
5	06	Alpes-Maritimes	27 038
6	94	Val-de-Marne	26 668
7	69	Rhône	23 658
8	59	Nord	22 657
9	33	Gironde	20 813
10	91	Essonne	16 044

Vérifications réalisées + analyse.

- *Conservation des effectifs* : la somme des comptes par département (757 263) coïncide avec le cumul affiché année par année par le script, et le nombre de départements représentés (**96** = 95 + Corse 2A/2B) confirme que le 20 corse a bien été éclaté et qu'aucun DOM n'est compté.
- *Plausibilité géographique* : le top 10 est dominé par **Paris et la petite couronne** (75, 93, 92, 94) plus les grandes métropoles (Bouches-du-Rhône/Marseille, Alpes-Maritimes/Nice,

Rhône/Lyon, Nord/Lille, Gironde/Bordeaux) — résultat attendu, le nombre d'accidents suivant la densité de population et de trafic urbain. Paris domine très nettement (81 016, soit $\sim 1,7 \times$ le 2^e).

- *Vérification de la carte* : ouverture du `.html`, contrôle visuel du placement des cercles sur les bons départements, de la proportionnalité des rayons (Paris le plus gros), et du contenu des popups (nom + effectif).

Commentaires. Le décompte est **brut** (volume d'accidents), non rapporté à la population ou au trafic : il mesure donc l'exposition absolue, pas un risque par habitant. Les départements urbains denses ressortent logiquement en tête. Une normalisation (accidents pour 100 000 habitants ou par véhicule-km) donnerait un classement très différent et ferait remonter des départements ruraux ou de transit — piste possible pour la question libre Q6.4.

Question 6.4 — Question libre

Rappel de la question. Question libre à définir, utilisant **au moins deux types de table** (parmi *caractéristiques*, *lieux*, *véhicules*, *usagers*) et **un type d’affichage non utilisé précédemment** (camembert exclu).

Question retenue :* Comment les accidents se répartissent-ils selon les conditions atmosphériques (atm**) croisées avec l’état de la surface de la route (**surf**) ? *** Les deux variables sont a priori** corrélées (la pluie mouille la chaussée, la neige l’enneige) mais pas confondues — on attend des cellules « hors diagonale » instructives (pluie sur chaussée déjà verglacée, temps normal sur chaussée mouillée résiduelle, etc.).

Travail avec l’agent — Claude Code

- *Choix des tables (contrainte ≥ 2) :* **atm** vit dans la table **caractéristiques**, **surf** dans la table **lieux**. Les deux sont jointes accident par accident via la clé **Num_Acc** : on lit d’abord caractéristiques pour mémoriser **Num_Acc** $\rightarrow$ **atm** (en ne retenant que la métropole via **normaliser_dep** sur **dep**), puis on parcourt lieux et, pour chaque **Num_Acc** présent dans ce dictionnaire, on incrémente la cellule (**atm**, **surf**). Filtrer dès la première table garantit que seuls les accidents métropolitains alimentent la matrice.
- *Choix de l’affichage (autre que les précédents, camembert exclu) :* une **heatmap** (carte de chaleur) **atm** $\times$ **surf**, affichage inédit dans le dossier (Q6.1 barres, Q6.2 barres groupées, Q6.3 carte folium). **seaborn** n’étant pas dans les dépendances du projet, la heatmap est tracée directement avec **matplotlib.imshow** (palette **YlOrRd**), chaque cellule étant annotée de son effectif.
- *Échelle de couleur :* la modalité (Normale, Normale) écrase tout (76 % des accidents). Une échelle **logarithmique** (LogNorm) a été choisie pour que les contrastes des modalités rares (neige, verglas, brouillard) restent lisibles — sans elle, toute la grille hors coin supérieur gauche apparaît d’une teinte uniforme.
- *Étiquettes :* les libellés sont **abrégés aux deux premiers mots** (abreger). Un seul mot ne suffisait pas (collisions « Pluie légère »/« Pluie forte » et « Temps éblouissant »/« Temps couvert ») ; deux mots distinguent toutes les modalités tout en gardant des axes courts.
- *Valeurs manquantes :* **atm** vide est ignoré (21 accidents) ; **surf** vide **ou égal à 0** (sentinelle « non renseigné » du BAAC) est ignoré (23 375 accidents). Les modalités sont figées dans l’ordre du référentiel (1 $\rightarrow$ 9) pour une grille stable d’une exécution à l’autre.

Fichier Python : python/question64.py.

Vérification (pandas) — python/verification64.py $\rightarrow$ results/verification64.txt :

```
>>> f = lieux.merge(carac_metropole, on="Num_Acc")
>>> recap = f.groupby(["atm","surf"]).size()
>>> recap # totaux exploites / ignores / couple dominant

      controle pandas
accidents_exploites 733867
      atm_ignores 21
      surf_ignores 23375
top_couple_effectif 560189

# conforme : identiques a reponse64.txt ; couple dominant 1 - Normale x 1 - Normale
```

Résultat : results/reponse64.png (heatmap) et results/reponse64.txt (trace). Sur **733**

867 accidents métropolitains exploitables (cumul 2005-2015), les couples les plus fréquents sont :

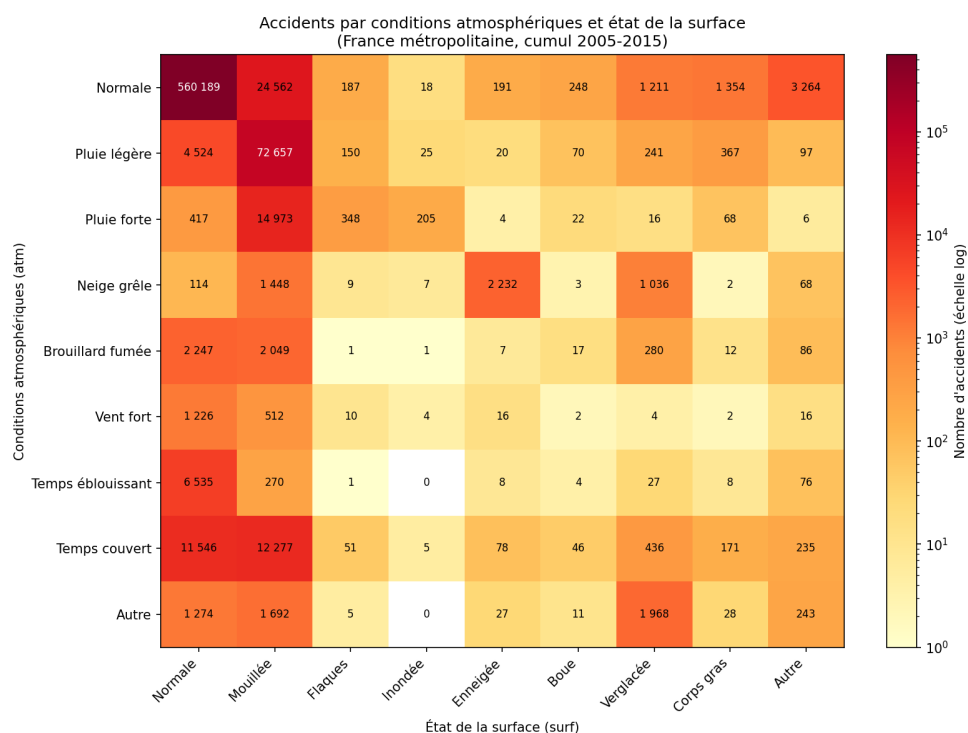


FIGURE 4 – Q6.4 — Heatmap des accidents selon les conditions atmosphériques (atm) × état de la surface (surf), échelle logarithmique (métropole, 2005-2015)

Conditions atm.	État surface	Nb accidents	Part
Normale	Normale	560 189	76,3 %
Pluie légère	Mouillée	72 657	9,9 %
Normale	Mouillée	24 562	3,3 %
Pluie forte	Mouillée	14 973	2,0 %
Temps couvert	Mouillée	12 277	1,7 %

Vérifications réalisées + analyse.

- *Conservation des effectifs* : le total exploité (733 867) = total métropolitain de Q6.3 (757 263) – les 23 396 accidents écartés pour atm/surf non renseignés (21 + 23 375). Cohérent.
- *Cohérence physique de la diagonale* : les cellules attendues ressortent nettement — **pluie légère × mouillée** (72 657, 2^e couple), **neige-grêle × enneigée** (2 232), **temps couvert/pluie × mouillée**. Le verglas apparaît surtout sur Neige grêle × Verglacée (1 036) et Autre × Verglacée (1 968, le « Autre » atmosphérique recouvrant vraisemblablement le grand froid sec).
- *Cellules hors diagonale instructives* : **6 535 accidents par temps éblouissant sur chaussée normale** (l'éblouissement n'altère pas la route mais la visibilité), et **24 562 accidents en temps « Normale » sur chaussée mouillée** (chaussée encore humide après la pluie alors que le ciel est redevenu clair) — ces deux poches justifient le croisement plutôt que deux histogrammes séparés.
- *Contrôle visuel* : ouverture du PNG, vérification que les libellés d'axes sont distincts, que l'échelle log rend les modalités rares visibles, et que les annotations chiffrées correspondent à la trace texte.

Commentaires. L'écrasante domination de (Normale, Normale) reflète simplement le fait que

la plupart des trajets se font par beau temps sur route sèche : la heatmap mesure des **volumes**, pas un sur-risque. Pour en déduire un risque relatif il faudrait rapporter chaque cellule à l'exposition (temps passé dans chaque condition météo/surface), donnée absente du BAAC. La lecture pertinente est donc **comparative entre conditions dégradées** : à conditions rares, la chaussée mouillée concentre l'essentiel des accidents pluvieux, et l'enneigement/verglas forme un bloc cohérent neige $\leftrightarrow$ surface, ce qui valide la qualité du croisement des deux tables.

Question 6.5 (*facultative*) — Carte animée par année

Rappel de la question. Reprendre la question 6.3 et en faire un **GIF animé** de la carte, année par année.

Travail avec l'agent — Claude Code

- *Choix de la bibliothèque de tracé* : folium produit du HTML interactif, inadapté à un GIF. Chaque image de l'animation a donc été tracée avec **matplotlib** (backend **Agg**, sans fenêtre), enregistrée en PNG, puis les 11 PNG ont été empilés en GIF avec **Pillow** (`Image.save(..., save_all=True, append_images=..., duration=800, loop=0)`). Ces deux dépendances sont déjà présentes dans le projet (matplotlib pour Q6.4, Pillow tiré par matplotlib).
- *Positions des cercles* : faute de fond de carte vectoriel, chaque département est placé à son **centroïde** (`CENTROIDES_DEPARTEMENTS`, couples *lat/lon* déjà utilisés en Q6.3). En traçant `x = lon`, `y = lat` avec `set_aspect("equal")`, la silhouette de l'Hexagone (Corse comprise) se reconstitue sans tracé de frontières.
- *Échelle comparable d'une image à l'autre — décision clé* : si l'on normalisait les rayons année par année, toutes les frames se ressembleraient et l'animation ne montrerait **aucune évolution**. On parcourt donc d'abord **toutes** les années pour calculer le **maximum global** (un département, une année : **8 591** accidents, Paris 2005), puis l'aire de chaque cercle est rapportée à ce maximum unique. Les cercles rétrécissent ainsi visiblement de 2005 à 2015.
- *Cadre fixe* : bornes des axes figées sur la métropole (`lon -5,5→10`, `lat 41→51,5`) et axes masqués, pour que les frames se superposent sans « sauter ».
- *Aire $\propto$ effectif* : le paramètre `s` de `scatter` étant une **aire** (points²), on le prend proportionnel au nombre d'accidents (« cercles proportionnels »).
- *Nettoyage* : les 11 PNG intermédiaires (`_carte_AAAA.png`) sont **supprimés** après l'assemblage ; seul le GIF subsiste.

Fichier Python : `python/question65.py`.

Vérification (pandas) — `python/verification65.py` → `results/verification65.txt` :

```
>>> recap = (caracteristiques["dep"].map(normaliser_dep)
... .value_counts()) # par annee
>>> recap # nb departements + total metropole / annee

annee departements total_metropole
2005 96 84525
2006 96 80309
2007 96 81272
2008 96 74487
2009 96 72315
2010 96 67288
2011 96 65024
2012 96 60437
2013 96 56812
2014 96 58191
2015 96 56603

# conforme : comptes par departement identiques annee par annee
```

Résultat : `results/reponse65.gif` — animation de **11 images** (2005→2015), 600×600 px, 800 ms par image, boucle infinie. Le décompte métropolitain par année décroît régulièrement :

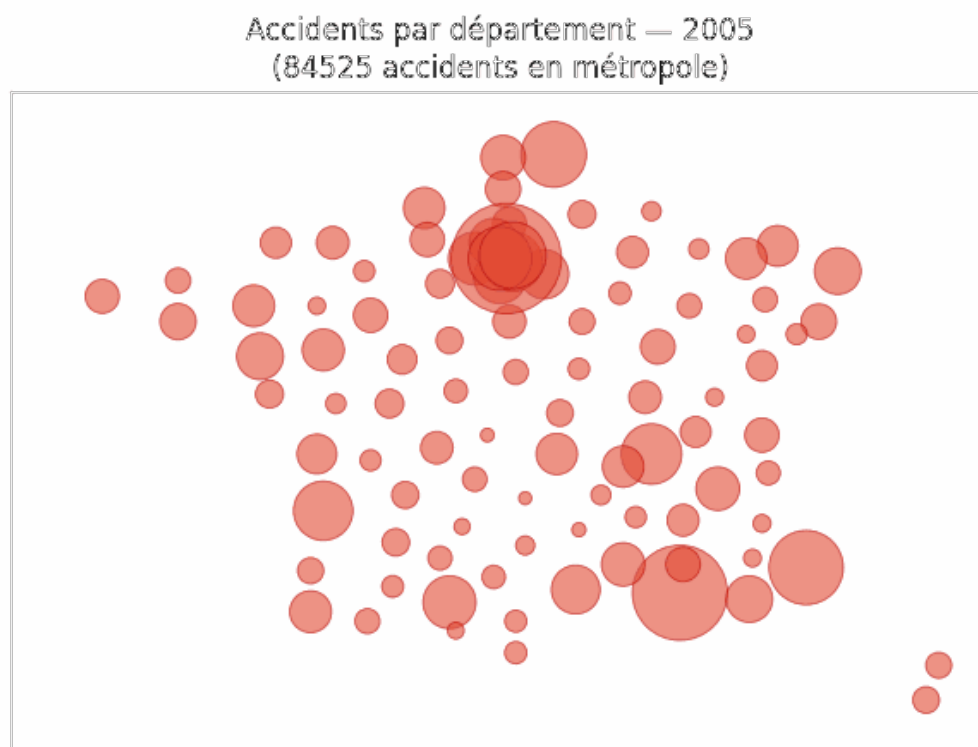


FIGURE 5 – Q6.5 — Première image (2005) du GIF animé `results/reponse65.gif` : accidents par département, cercles proportionnels (métropole)

Année	Accidents métropole	Année	Accidents métropole
2005	84 525	2011	65 024
2006	80 309	2012	60 437
2007	81 272	2013	56 812
2008	74 487	2014	58 191
2009	72 315	2015	56 603
2010	67 288		

Vérifications réalisées + analyse.

- *Conservation des effectifs* : la somme des 11 totaux annuels (757 263) coïncide avec le cumul métropolitain de Q6.3, et chaque total annuel est réaffiché par le script — aucune ligne perdue ni dupliquée.
- *Structure du GIF* : relecture du fichier avec Pillow → `n_frames = 11`, `duration = 800`, `loop = 0`, taille 600×600 ; les 11 PNG temporaires ont bien disparu du dossier `results/`.
- *Contrôle visuel* : ouverture de la première (2005) et de la dernière (2015) frame. La silhouette de la France est reconnaissable, **Paris** (75) forme le gros amas sombre central, la **Corse** apparaît détachée en bas à droite, et les cercles **rétrécissent nettement** entre 2005 et 2015 — l'animation traduit donc bien la baisse continue des accidents.

Commentaires. L’animation met en évidence la **baisse tendancielle** du nombre d’accidents corporels sur la décennie (−33 % entre 2005 et 2015), cohérente avec le renforcement des politiques de sécurité routière. La représentation reste **volumétrique** (mêmes réserves qu’en Q6.3 : pas de normalisation par population/trafic) et **schématique** (centroïdes plutôt qu’un vrai fond cartographique) : son intérêt est la lecture dynamique de l’évolution, pas la précision géographique. Un fond **geopandas**/contours départementaux donnerait un rendu choroplèthe plus fin, au prix d’une dépendance supplémentaire non requise par le sujet.

Conclusion / synthèse

Qualité des données. L’intégrité référentielle du jeu BAAC 2005-2015 (Groupe 1) est globalement très bonne. Le lien *usager* → *véhicule* est quasi parfait (22 violations sur 1 742 583 usagers, soit 0,0013 %), et le lien inverse *véhicule* → *usager* ne présente que 1,47 % de véhicules sans usager (19 580 cas), majoritairement explicables par les conducteurs en fuite. Les contrôles facultatifs *lieu* ↔ *accident* (Q2.3) et *accident* → *lieu unique* (Q2.4) confirment cette cohérence. Le seul attribut véritablement fragile au sens de la saisie est le sens de circulation (**senc**) ; les taux d’absence élevés de **obs/obsm/choc/occutc** (Q4.1) traduisent surtout des « sans objet » légitimes (code 0) et non des défauts de remplissage.

Cohérence inter-attributs. Les incohérences logiques recherchées (Q5) restent marginales : luminosité × heure (Q5.1), obstacle mobile × point de choc (Q5.2) et années de naissance impossibles (Q5.3, 41 cas sur 1,74 M) ne remettent pas en cause l’exploitabilité du jeu. Ces contrôles valident la fiabilité d’ensemble plus qu’ils ne révèlent des anomalies massives.

Enseignements analytiques. Les visualisations (Q6) dégagent des structures conformes aux connaissances de la sécurité routière : distribution horaire bimodale calée sur les déplacements domicile-travail (pic 17h-18h), surmortalité des hommes jeunes (15-29 ans, 75,7 % des tués), concentration des accidents dans les départements urbains denses (Paris ≈ 1,7 × le 2^e), et baisse tendancielle de −33 % du nombre d’accidents corporels entre 2005 et 2015.

Limites. Tous les décomptes géographiques et temporels sont **volumétriques** : ils mesurent une exposition absolue, non un risque rapporté à la population, au trafic ou aux véhicules-km — données absentes du BAAC. Les cartes (Q6.3/6.5) reposent sur des **centroïdes** départementaux, schématiques par rapport à un vrai fond cartographique.

Pistes d’amélioration. Normaliser les effectifs (accidents pour 100 000 habitants), enrichir avec un fond **geopandas** pour des cartes choroplèthes, et croiser gravité × infrastructure pour cibler les configurations les plus meurtrières.